The proof run — one funded afternoon that turns three rows from BUILT to PROVEN

> For the founder. Written 5 Sep 2026. Everything below is a click you make or a message you send; nothing here moves money without you. Total: ~$32 USDC and about two hours. Do it only when the product feels right (`§0` of the plan) — and if it does, do it before **Sat 19:00 PKT** so the receipts are in the Future Caribbean package; after that it still serves STRK20 (Mon 04:59).

What one run proves

The run does	Which claim it turns true
A grant launched from the composer, priced in **J$**	the first milestone grant ever paid in production · "priced in 14 currencies" · `/outcomes` gets its first corridor reading
Launched on the **Starknet** rail	a private payout on the declared privacy class, with hashes for `strk20.json` · the private leg drawn on `/proof`
A recipient onboarded by a **Telegram link**	walletless recipients: first real invite, first real chat wallet
Tranche 1 paid → the recipient **takes the advance** from their record	the first advance disbursed
Tranche 2 paid → the **waterfall** repays it	the first advance repaid — capital out, capital back in

Before you start (10 minutes)

1. **Second Telegram account** on your phone (or a friend). It will be the recipient.
2. **USDC on Starknet mainnet** in the wallet you will launch from: **~$25** (the grant) — the composer shows the exact J$→USDC figure at the stamped rate before you fund anything.
3. **~$5 USDC on the operator's Starknet account** — the advance pot: `0x46a1747d854e74e5082c3215841b26dcff182a6a6fd7a1f83c3e1d045996101` (read 5 Sep 03:20 PKT: USDC **0**, STRK 27.3 — gas is covered, the pot is empty). `ADVANCE_SELF_SERVE=1` is already armed on prod (max $5, multiple 1×, waterfall 50%); the button appears on a record once the pot has funds.
4. **~$2 USDC** to the GOAT fee wallet `0x0deF3D4124D0cD1708aEFFE6c1BC8182342a44D6` (read 5 Sep: USDC **0**, BTC gas 0.000016 — enough). Sixteen $0.10 operator-fee transfers are pending on it ($1.60); they settle on the next sweep once it holds USDC and show on `/explorer` as "How Sage earns".

Step 0 — the treasury move (10 minutes, ~$15 USDC on GOAT) — do this first

Tonight the board has **zero open mainnet slots**: every live campaign is fully paid out. The film's marketplace row, the verify card and "What Sage would do next" all need open PUBLIC work, and the product's own answer is the treasury: open `/workspace/autopilot`, read the move the rehearsal shows (priced, with its reason — it is the real decision, sized as if the treasury held $15), then fund the treasury with that amount from your Ethereum wallet on GOAT. Sage proposes the move with its veto ring, launches it after the window, and the missions it buys are public — so the door ("prove you are one person") appears on real mainnet work, on the marketplace and on the board. That is scenes 4 and 7.

Dry walk at $0 first, if you want: `/c/launch-yara-garden-343gb2` is a listed Metis-Sepolia testnet campaign with 10 open $0.10 slots — connect any wallet or **continue with email** (Sage keeps the wallet), the door card asks for the World ID proof, verify once ("one slot on this work is yours"), submit an observation, and a testnet payout settles. Same code path, no real money. Then open your record page: **Your wallet** shows the balance and a gasless **Withdraw** (you sign, Sage pays the gas).

The run (about 90 minutes, mostly waiting for the recipient)

1 · Compose the grant (5 min). Signed in at https://sagepays.xyz/launch/direct, or say it to @sagedeputybot in one sentence — either door compiles the same plan:

> *Give a market seller J$10,000 in two equal parts — half when her catalogue page is online with her wallet address on it, half when she posts her first customer review.*

You will see: two milestones, J$5,000 each, the stamped rate and source, the USDC total, and the verification for each (a published page carrying the recipient's wallet). Choose **Starknet** as the rail and **invite-only** (so only your recipient can claim). Approve. Nothing has moved.

2 · Fund and launch (5 min). Fund the vault from your Starknet wallet with the exact USDC the plan states. The campaign goes live; its board is `/c/<id>`. The row now says **J$10,000 → $X @ rate** on the board, and the same on `/marketplace` if you made it public (don't — invite-only).

3 · Invite the recipient (2 min). In @sagedeputybot: *"invite a recipient to "* — it answers with a one-time `t.me/sagedeputybot?start=rcp_…` link. Send it to the second account. Opening it mints their wallet; the chat is their account from then on.

4 · Tranche 1 — the catalogue page (20 min). From the second account, publish a public page (a dev.to post, a Notion page, a gist) with a few catalogue items and **the wallet address the bot shows them**. Then in the bot: *"submit for the catalogue milestone"*. Sage fetches the page, verifies the wallet marker, judges, and — because the campaign is invite-only — **pays at once**: the vault releases, Sage escrows behind `poseidon(secret)`, and the bot sends the recipient a one-time claim link. Open `/proof/<tx>`: the receipt now shows **The private leg** with the escrow transaction and "waiting to be collected". Have them collect (the link pays the gas). The receipt flips to "collected"; who collected it is not on the ledger.

5 · The advance (5 min). Open the recipient's record: `/record/<their wallet>`. With `ADVANCE_SELF_SERVE=1` on, the record shows the advance capacity (the lender's multiple × verified monthly inflow — arithmetic, not a score) and one button. They take it; the pot escrows it to a claim link; they collect. `/lender?wallet=<their wallet>` shows the facility and its terms.

6 · Tranche 2 — the review (20 min). They publish the customer review page with the wallet marker and submit it. This payout settles as **two legs** on Starknet: the pot's slice (the waterfall repayment, at the published fraction) and the recipient's remainder. `/record` shows the advance **repaid**; `/outcomes` shows "Measured: 2 obligations priced in JMD … benchmarked against Jamaica's 3.59%"; `/explorer` lists both settlements.

7 · Tell me the hashes. I put the private-route transactions into `strk20.json`, refresh every number in the package from the live ledger, and update the film script's on-screen figures.

If something refuses

Every refusal names its reason. The two you might see: *"the page doesn't carry your wallet"* — the address on the page must be the one the bot showed the recipient, exactly; and *"held for the finalization window"* — only on public campaigns, not this one. Nothing on this path spends model budget on a refused page, and a refused submission can be resubmitted.